

从 Delta Attention 到 Gated Delta Attention, 增加的核心能力是:

Delta Attention 只能沿当前 key 的方向修改记忆; Gated Delta Attention 在此基础上增加一个遗忘门 α_t , 让整个旧状态可以随时间衰减。可以把两者分别理解成:

- β_t : 当前 key 对应的位置要改多少;
- α_t : 整个旧记忆要保留多少。

一、先回顾 Delta Attention

继续使用统一的矩阵方向:

$$k_t, q_t \in \mathbb{R}^{d_k}$$

$$v_t \in \mathbb{R}^{d_v}$$

$$S_t \in \mathbb{R}^{d_k \times d_v}$$

Delta Attention 的状态更新为:

$$v_t^{\text{old}} = k_t S_{t-1}$$

$$u_t = \beta_t (v_t - v_t^{\text{old}})$$

$$S_t = S_{t-1} + k_t^\top u_t$$

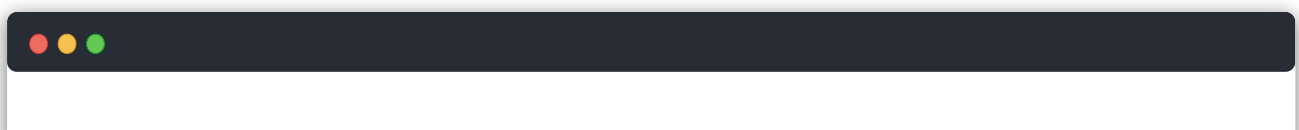
合并起来:

$$S_t = S_{t-1} + \beta_t k_t^\top (v_t - k_t S_{t-1})$$

输出为:

$$o_t = q_t S_t$$

代码就是:



```
old_value = k_t @ state
delta = beta_t * (v_t - old_value)
state = state + outer(k_t, delta)
out = q_t @ state
```

二、Delta Attention 能修改什么？

Delta Rule 只能沿 k_t 方向修改状态。将公式展开：

$$\begin{aligned} S_t &= S_{t-1} + \beta_t k_t^\top v_t - \beta_t k_t^\top k_t S_{t-1} \\ &= (I - \beta_t k_t^\top k_t) S_{t-1} + \beta_t k_t^\top v_t \end{aligned}$$

所以：

$$S_t = \underbrace{(I - \beta_t k_t^\top k_t) S_{t-1}}_{\text{沿 } k_t \text{ 方向擦除旧信息}} + \underbrace{\beta_t k_t^\top v_t}_{\text{沿 } k_t \text{ 方向写入新信息}}$$

其中：

$$k_t^\top k_t$$

是一个秩为 1 的投影矩阵，只影响状态中与 k_t 方向相关的部分。因此 Delta Attention 擅长：

把“当前 key 对应的旧值”替换成新值。但它不擅长：在没有提供对应新 key-value 的情况下，让一批旧记忆整体消失。

三、Delta Attention 为什么还需要全局遗忘？

假设状态里存有三条记忆：

$$S = k_1^\top v_1 + k_2^\top v_2 + k_3^\top v_3$$

当前来了一个新的 key k_4 。Delta Rule 主要修改 k_4 方向：

$$S' = S + \beta_4 k_4^\top (v_4 - k_4 S)$$

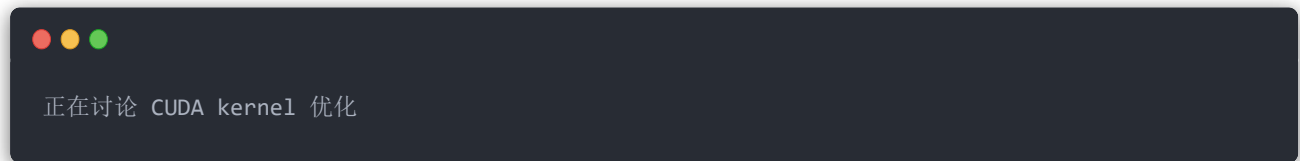
如果 k_4 与 k_1, k_2, k_3 大致正交, 那么:

$$k_4 k_1^\top \approx 0$$

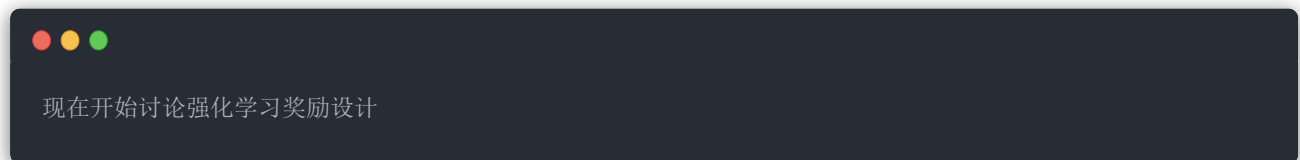
$$k_4 k_2^\top \approx 0$$

$$k_4 k_3^\top \approx 0$$

此时 Delta Rule 几乎不会删除 v_1, v_2, v_3 。这在某些场景下会有问题。例如上下文从:



突然切换为:



模型可能希望整体降低前一个话题状态的影响, 而不是必须逐个找到所有旧 key, 再逐项覆盖。Delta Rule 没有一个直接的“清空旧上下文”操作。

四、Gated Delta Attention 增加 α_t

Gated Delta Attention 在旧状态上增加一个衰减门:

$$\tilde{S}_{t-1} = \alpha_t S_{t-1}$$

其中:

$$0 < \alpha_t \leq 1$$

然后在衰减后的状态上执行 Delta Rule:

$$v_t^{\text{old}} = k_t \tilde{S}_{t-1}$$

$$u_t = \beta_t (v_t - v_t^{\text{old}})$$

$$S_t = \tilde{S}_{t-1} + k_t^\top u_t$$

合并起来：

$$S_t = \alpha_t S_{t-1} + \beta_t k_t^\top (v_t - k_t \alpha_t S_{t-1})$$

进一步展开：

$$S_t = \alpha_t (I - \beta_t k_t^\top k_t) S_{t-1} + \beta_t k_t^\top v_t$$

这就是常见的 Gated Delta Rule。有些论文采用转置的状态定义，可能写成：

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

两者只是矩阵方向不同。

五、公式中每一项是什么意思？

Gated Delta Rule：

$$S_t = \alpha_t (I - \beta_t k_t^\top k_t) S_{t-1} + \beta_t k_t^\top v_t$$

可以拆成三步。

1. 全局衰减

$$\alpha_t S_{t-1}$$

让所有旧记忆都乘上 α_t 。

- $\alpha_t = 1$ ：完全保留旧状态；
- $\alpha_t = 0.9$ ：保留 90%；
- $\alpha_t = 0.1$ ：大幅遗忘；
- $\alpha_t \rightarrow 0$ ：近似清空旧状态。

2. 局部擦除

$$-\alpha_t \beta_t k_t^\top k_t S_{t-1}$$

在全局衰减后的状态里，进一步擦除当前 k_t 方向上的旧值。

3. 写入新值

$$\beta_t k_t^\top v_t$$

沿当前 k_t 方向写入目标值 v_t 。因此完整过程是：

先让所有旧记忆整体衰减

↓

再读取当前 key 方向还剩什么

↓

擦除当前 key 方向的残余旧值

↓

写入当前目标 value

六、 α_t 和 β_t 到底有什么区别？

这是理解 Gated Delta Attention 最重要的地方。

门	作用范围	主要功能
α_t	整个状态 S	全局保留或遗忘旧记忆
β_t	当前 k_t 方向	控制当前关联的擦除和写入强度

可以用一个白板类比：

- α_t ：让整块白板上的所有字变淡；
- β_t ：在某个具体位置擦掉旧字，再写入新字。所以：

$$\alpha_t = \text{全局遗忘门}$$

$$\beta_t = \text{局部更新门}$$

七、从 Delta 到 Gated Delta 的代码变化

Delta Attention

```
old_value = k_t @ state

delta = beta_t * (
    v_t - old_value
)

state = state + outer(
    k_t,
    delta,
)

out = q_t @ state
```

Gated Delta Attention



```

# 1. 全局衰减
state = alpha_t * state

# 2. 读取衰减后, 当前 key 方向的旧值
old_value = k_t @ state

# 3. 计算需要补写的差值
delta = beta_t * (
    v_t - old_value
)

# 4. 沿当前 key 方向写回
state = state + outer(
    k_t,
    delta,
)

# 5. query 读取更新后的状态
out = q_t @ state

```

核心变化看起来只有一行：

```

state = alpha_t * state

```

但这行使模型拥有了主动清理整个历史状态的能力。

八、完整的逐 token 实现

输入形状：

```

q:      [B, H, T, Dk]
k:      [B, H, T, Dk]
v:      [B, H, T, Dv]
alpha:  [B, H, T, 1]
beta:   [B, H, T, 1]
state:  [B, H, Dk, Dv]

```

参考代码:

```
import torch
import torch.nn.functional as F

def gated_delta_attention_recurrent(
    q,
    k,
    v,
    alpha,
    beta,
    state=None,
):
    """
    q:      [B, H, T, Dk]
    k:      [B, H, T, Dk]
    v:      [B, H, T, Dv]
    alpha:  [B, H, T, 1]
    beta:   [B, H, T, 1]

    state:  [B, H, Dk, Dv]
    """

    q = F.normalize(
        F.silu(q),
        p=2,
        dim=-1,
    )

    k = F.normalize(
        F.silu(k),
        p=2,
        dim=-1,
    )

    B, H, T, Dk = q.shape
    Dv = v.shape[-1]

    if state is None:
        state = torch.zeros(
```



```

        B,
        H,
        Dk,
        Dv,
        device=q.device,
        dtype=q.dtype,
    )

    outputs = []

    for t in range(T):
        q_t = q[:, :, t]
        k_t = k[:, :, t]
        v_t = v[:, :, t]

        alpha_t = alpha[:, :, t]
        beta_t = beta[:, :, t]

        # 1. 全局遗忘
        state = alpha_t.unsqueeze(-1) * state

        # 2. 读取当前 key 方向的旧值
        old_value = (
            k_t.unsqueeze(-2) @ state
        ).squeeze(-2)

        # 3. Delta Rule
        delta = beta_t * (
            v_t - old_value
        )

        # 4. 外积写入
        update = (
            k_t.unsqueeze(-1)
            @ delta.unsqueeze(-2)
        )

        state = state + update

        # 5. query 读取状态
        o_t = (
            q_t.unsqueeze(-2) @ state
        ).squeeze(-2)

```

```
outputs.append(o_t)

output = torch.stack(
    outputs,
    dim=2,
)

return output, state
```

九、用两条独立记忆举例

假设状态中保存两条正交关联：

$$S_{t-1} = k_1^\top v_1 + k_2^\top v_2$$

其中：

$$k_1 k_2^\top = 0$$

现在新的输入对应 k_1 ，希望把 v_1 改成 v'_1 。

1. 普通 Delta Attention

更新：

$$S_t = S_{t-1} + \beta k_1^\top (v'_1 - k_1 S_{t-1})$$

取：

$$\beta = 1$$

因为：

$$k_1 S_{t-1} = v_1$$

所以：

$$S_t = k_1^\top v'_1 + k_2^\top v_2$$

结果：

- v_1 被替换成 v'_1 ;
 - v_2 完全不变。
-

2. Gated Delta Attention

假设：

$$\alpha = 0.5, \quad \beta = 1$$

先衰减：

$$\tilde{S} = 0.5k_1^\top v_1 + 0.5k_2^\top v_2$$

用 k_1 读取：

$$k_1 \tilde{S} = 0.5v_1$$

计算差值：

$$u = v'_1 - 0.5v_1$$

写回：

$$S_t = \tilde{S} + k_1^\top u$$

展开：

$$S_t = k_1^\top v'_1 + 0.5k_2^\top v_2$$

结果：

- 当前 k_1 对应的值被完整更新为 v'_1 ;
- 与当前写入无关的 v_2 衰减到原来的 50%。这就是两个门不同的作用：

$$\beta = 1$$

确保当前关联被改写；

$$\alpha = 0.5$$

让其他历史记忆逐渐淡化。

十、为什么先衰减，再读取旧值？

这里的顺序很重要。正确逻辑是：

```
state = alpha * state
old = k @ state
delta = beta * (v - old)
state = state + outer(k, delta)
```

也就是先得到：

$$\tilde{S} = \alpha S_{t-1}$$

再计算：

$$v^{\text{old}} = k\tilde{S}$$

因为我们要修改的是“衰减以后仍然残留的旧值”。如果先读取原始状态：

```
old = k @ state
state = alpha * state
delta = beta * (v - old)
state += outer(k, delta)
```

当 $\beta = 1$ 时，读取结果可能不是目标值。假设：

$$kS_{t-1} = v_{\text{old}}$$

错误顺序更新后：

$$S_t = \alpha S_{t-1} + k^\top (v - v_{\text{old}})$$

再读取：

$$\begin{aligned} kS_t &= \alpha v_{\text{old}} + v - v_{\text{old}} \\ &= v - (1 - \alpha)v_{\text{old}} \end{aligned}$$

它不等于 v 。正确顺序则是：

$$S_t = \alpha S_{t-1} + k^\top (v - \alpha kS_{t-1})$$

读取：

$$kS_t = \alpha v_{\text{old}} + v - \alpha v_{\text{old}} = v$$

所以必须：

先衰减，再读残余，再写差值

十一、 α_t 是怎么生成的？

一般由输入隐藏状态预测：

$$\alpha_t = f_\alpha(x_t)$$

最简单可以使用 sigmoid：

$$\alpha_t = \sigma(W_\alpha x_t + b_\alpha)$$

代码：

```
alpha = torch.sigmoid(
    self.alpha_proj(x)
)
```

但为了让模型初始化时倾向于长记忆，也可能使用指数形式：

$$\alpha_t = \exp(-\text{softplus}(a_t))$$

因为：

$$\text{softplus}(a_t) > 0$$

所以：

$$0 < \alpha_t < 1$$

代码：

```
log_alpha = -F.softplus(
    self.alpha_proj(x)
)

alpha = torch.exp(log_alpha)
```

使用 `log_alpha` 还有一个工程优势：长序列中多个 α 连乘时，可以在 $\log$ 空间中使用累加，数值更稳定。

十二、连续多步后的记忆会怎样？

假设在时刻 i 写入一条记忆，后面没有相同 key 的定点改写。经过 $i+1, \dots, t$ 的全局门后，该记忆会乘上：

$$\alpha_{i+1}\alpha_{i+2}\cdots\alpha_t$$

因此它在时刻 t 的保留比例是：

$$\prod_{j=i+1}^t \alpha_j$$

例如每一步：

$$\alpha = 0.9$$

经过 10 步：

$$0.9^{10} \approx 0.349$$

只剩下约 34.9%。经过 100 步：

$$0.9^{100} \approx 2.66 \times 10^{-5}$$

几乎完全消失。如果：

$$\alpha = 0.999$$

经过 100 步：

$$0.999^{100} \approx 0.905$$

仍保留约 90.5%。所以模型可以通过控制 α_t ，学习不同的记忆时间长度。不过，普通 Gated DeltaNet 的单个 head 通常共享一个标量 α_t ，同一个 head 内的所有状态通道以相同比例衰减。这也是后续 KDA 要引入更细粒度通道门控的原因。

十三、与 Mamba 门控的关系

如果暂时不考虑 Delta 修正：

$$\beta_t k_t^\top k_t$$

Gated Delta Rule 可以简化为：

$$S_t = \alpha_t S_{t-1} + k_t^\top v_t$$

这与一般的门控递归状态非常相似：

$$h_t = \alpha_t h_{t-1} + b_t x_t$$

因此可以理解为：

- Mamba 类门控提供整体状态衰减；
- Delta Rule 提供 key 方向上的精确更新；
- Gated DeltaNet 把两者结合起来。即：

十四、为什么 Gated Delta Prefill 更难并行？

普通 Delta Rule 已经有状态依赖：

$$S_t = f_t(S_{t-1})$$

Gated Delta 又增加：

$$S_t = \alpha_t A_t S_{t-1} + B_t$$

其中：

$$A_t = I - \beta_t k_t^\top k_t$$

把它展开到多个时间步：

$$S_t = \alpha_t A_t \alpha_{t-1} A_{t-1} \cdots \alpha_1 A_1 S_0 + \cdots$$

一个较早时刻的状态写入，传播到当前时刻时，会乘上累计衰减：

$$\gamma_t = \prod_{j=1}^t \alpha_j$$

因此 chunk-wise 实现通常会计算：

$$\gamma_t$$

或者在 log 空间计算：

$$\log \gamma_t = \sum_{j=1}^t \log \alpha_j$$

代码形式类似：

```
log_alpha_cumsum = torch.cumsum(  
    log_alpha,
```



```
        dim=-1,
    )

    gamma = torch.exp(
        log_alpha_cumsum
    )
```

块内某个位置 i 的信息传播到位置 j 时，需要乘：

$$\frac{\gamma_j}{\gamma_i}$$

这就是 Gated DeltaNet chunk-wise 代码里经常出现累计衰减、比例项或 `g_cumsum` 的原因。Decode 每次只有一个 token，不需要做这些并行重参数化，直接递归更新即可。

十五、从 Delta 到 Gated Delta 的核心对比

项目	Delta Attention	Gated Delta Attention
状态更新	$S + k^\top \beta(v - kS)$	先 αS ，再执行 Delta 更新
α	无	全局遗忘门
β	局部更新门	局部更新门
定点覆盖	支持	支持
整体遗忘	不支持	支持
话题切换	只能靠后续 key 逐渐覆盖	可以快速降低整个旧状态
长期记忆	旧关联可能一直残留	由累计 $\prod \alpha_t$ 控制
Prefill	需要 Delta chunk 算法	还要处理累计衰减
Decode 状态	固定大小	固定大小

最简代码对比：

```
# Delta Attention
old = k @ state
delta = beta * (v - old)
state = state + outer(k, delta)
```

```
# Gated Delta Attention  
state = alpha * state  
old = k @ state  
delta = beta * (v - old)  
state = state + outer(k, delta)
```

一句话总结：

Delta Attention 解决 “当前地址里的旧内容怎么替换” ； Gated Delta Attention 进一步解决 “那些没有被当前 key 访问到的陈旧内容怎么逐渐忘掉” 。